

Report for Laboratory 5: Introduction to Interrupts

Julian Soh

Department of Mechanical Engineering, Saint Martin's University
ME/EE 477—Embedded Real-Time Computing for Mechanical Engineers

June 21, 2026

Abstract. This lab explored the use of interrupts and how interrupts are handled by the FPGA of the myRIO-1900. The interrupts will respond to an external signal, which is the trigger from an SPDT (Single Pole, Double Throw) switch. This lab also examined the mechanical properties of the SPDT switch with respect to the electrical signals it affects and how the use of a flip-flop to store the state of a switch will help smooth the transition and prevent noise from affecting the system. The flip-flop circuit used in this lab was a NAND debouncing circuit.

1 Introduction

We connected an SPDT switch to the myRIO DIO0 on Connector A, as shown in Figure 1. Connector A on the myRIO is pin 11. Other pins that were identified and used in this lab include:

- DIO0 (pin 11)
- GND (pin 12)
- +3.3V (pin 33)

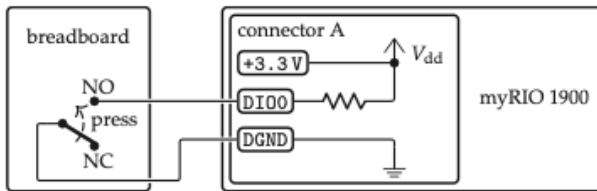


Figure 1: Connecting SPDT directly to myRIO DIO0. Picone et al. (2024).

The objectives of this lab were:

1. To implement a system with an external interrupt
2. To program an interrupt service routine
3. To program multiple threads
4. To prototype a digital logic circuit
5. To debounce a mechanical switch

2 Materials and Methods

2.1 Materials

The materials used in this experiment were:

- Windows 10 computer with myRIO support for C and Eclipse IDE.
- NI myRIO-1900
- Rigol DHO914S Digital Oscilloscope with 25Mhz function generator

- Rigol 712 DC power supply (to supply power to the NAND Integrated Circuit (IC))¹
- breadboard
- 1x NAND Integrated Circuit (IC) - SN7438N
- 4x 2.2 kΩ resistors
- Cables:
 - BNC with alligator clips
 - jumper cables

2.2 Using a debouncing circuit

After experimenting with the SPDT switch connected directly to the myRIO and observing the signal characteristics on the oscilloscope, we implemented a debouncing circuit using a NAND gate to smooth the transitions and prevent noise from affecting the system. The schematic of the debouncing circuit is shown in Figure 2.

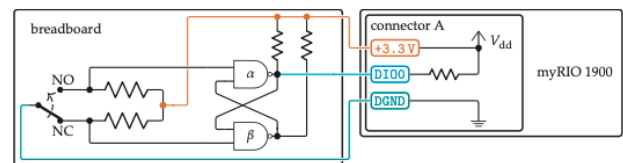


Figure 2: Schematic of the debouncing circuit using a NAND gate. Picone et al. (2024).

The actual implementation of the debouncing circuit is shown in Figure 3. The NAND IC was powered by the Rigol 712 DC power supply set at +3.3V with the ground connected to the myRIO. Pin 11 of the myRIO is connected to the output of the first NAND gate (pin 3).

¹This was optional because we could have drawn power from the function generator of the oscilloscope or from the myRIO. But because we used this, we had to make sure the ground from this power supply is tied to the ground of the myRIO and the entire circuit so as to establish a common ground.

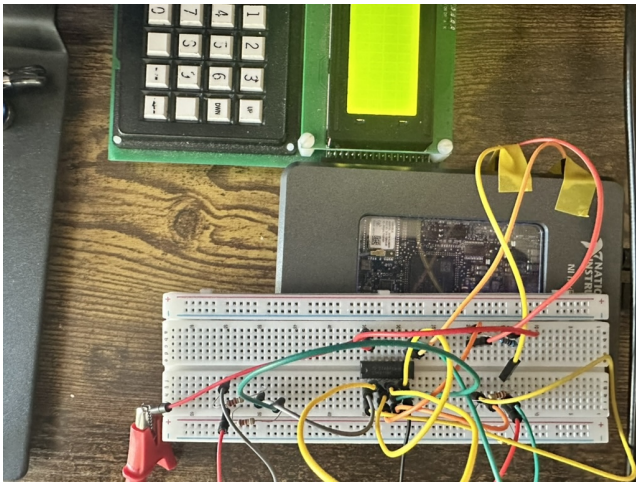


Figure 3: Actual implementation of the debouncing circuit. Picone et al. (2024).

3 Laboratory Exploration and Evaluation

The complete source code for the lab is available on GitHub at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab5/main.c.

A video demonstration of the lab is available at https://github.com/juliansoh/ME477/blob/main/Reports/Lab5/videos/IMG_0052.MOV.

3.1 Problem L5.1

Write and debug the `main()` and `DI_ISR()` functions described in section L5.2.

See code at https://github.com/juliansoh/ME477/blob/main/workspace/Julian_lab5/main.c.

3.2 Problem L5.2

Provide an interrupt signal by connecting the SPDT switch on the circuit bread board to DIO0 of connector A, as shown in figure 5.15. Notice that pressing the switch causes a falling edge signal to be applied to the DIO0 input. Try your program now. What happens? This undesirable phenomenon is caused by the “bounce” of the mechanical switch. See Figure 1.

Adjust the oscilloscope to trigger only on a high-to-low transition of the IRQ signal. Observe several of the transitions. Typically, what length of time is required for the transition to settle at the low level? How many triggers occur during the settling?

When the SPDT switch was pressed, as expected, DIO0 detected the signal change from high to low, thus triggering the interrupt. The Interrupt Service Routine (ISR) was then executed. However, we noticed that it was not a clean transition, but rather a series of rapid transitions due to the switch bouncing. As a result, the interrupt was triggered

multiple times, each triggering the ISR, resulting in printing “Interrupt” on the LCD numerous times as shown in Figure 4 and Figure 5.



Figure 4: LCD display showing multiple ISR execution (1 of 2).

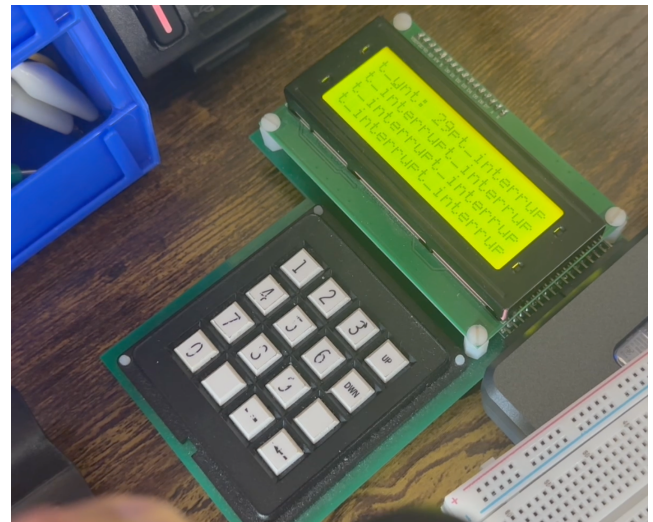


Figure 5: LCD display showing multiple ISR execution (2 of 2).

Connecting the oscilloscope, we confirmed the signal went from HIGH to LOW (+3.3V to 0V) and also observed the presence of the switch bounce based on the multiple triggers observed during the settling time, as shown in Figure 6 and Figure 7. Therefore, instead of one press, the myRIO interpreted the signals as multiple presses.

From what we observed, there were usually two triggers during settling, although there were a few instances where three or more triggers occurred. The multiple firing of the ISR shown in Figure 5 is an example of when more than two triggers occurred during the settling time.

We used the cursor to measure the time between the initial trigger and the final stable state of the signal. As shown

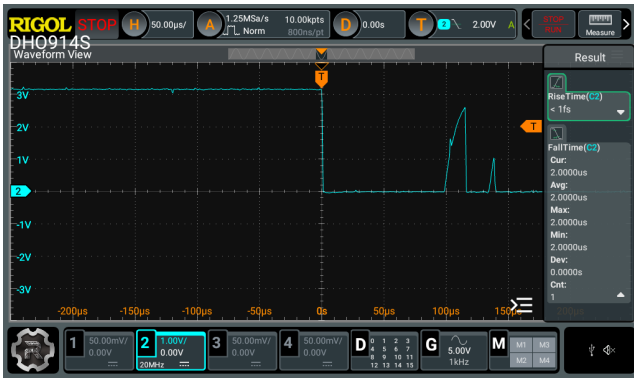


Figure 6: Observing switch bounce from the oscilloscope (1 of 2).

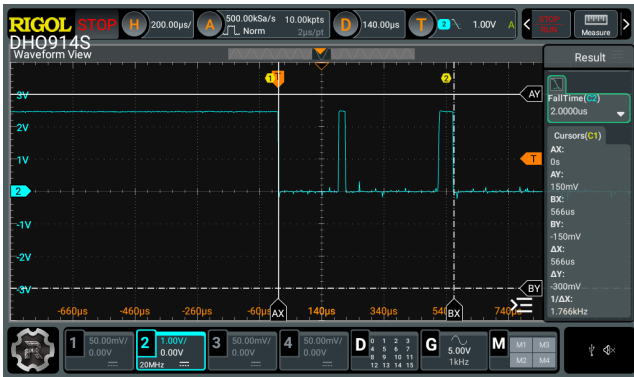


Figure 7: Observing switch bounce from the oscilloscope (2 of 2) WITH measurements of settling time ΔBX .

in Figure 7, the settling time was approximately $566\mu s$ (See the ΔBX reading in the right pane of the oscilloscope). Measuring the settling time for the bounce captured in Figure 4 yielded a smaller number of $\Delta BX = 141.5\mu s$, as shown in Figure 8. Nonetheless, in both cases, we noted that the settling times were very short and in rapid succession.



Figure 8: Another measurement of settling time ΔBX .

3.3 Problem L5.3

Correct the problem by replacing the switch in figure 5.15 with the debouncing circuit shown in figure 5.16. This circuit incorporates a TTL Quad open-collector NAND IC,

(e.g., a SN7438N.) Caution: Be certain that V_{dd} and GND are connected to the chip before wiring the rest of the circuit. Try your program again.

The program now only fires the interrupt once per switch press, and as such, the ISR is executed only once per switch press. The oscilloscope showed a clean, stable signal transition from HIGH to LOW without the multiple triggers observed previously, as can be seen in Figure 9.



Figure 9: Oscilloscope reading of the debounced signal.

3.4 Problem L5.4

Consider the switch-debouncing circuit of figure 5.13 and the traces of the outputs/states Q and Q^* shown in figure 5.14. Explain why each Q and Q^* state transition occurs and how this debounces the circuit that you built in lab problem L5.3.

Look at the two circuits we implemented in the lab, as portrayed by the schematic diagrams in Figure 10.

3.4.1 Without Debouncing Circuit

Prior to implementing the debouncing circuit, when the switch was pressed, as depicted in Figure 10(a), with P connected to DIO0, the state of P oscillated between 1 and 0 (high and low) when the switch is pressed because it took time to stabilize the switch and during that time, it oscillated between NO and NC. This oscillation was captured by both the oscilloscope and the FPGA through DIO0.

3.4.2 With Debouncing Circuit - Initial Conditions

From Figure 10(b), this was the initial condition when the switch was in the "not-pressed" state.

$$\bar{S} = 1$$

$$\bar{R} = 0$$

The reason was because V_{dd} flowed through the pull-up resistor then through NO and to $\bar{S}$, thereby making $\bar{S} = 1$. The switch was closed at NC so V_{dd} flowed through the pull-down resistor to GND, thus $\bar{R} = 0$. Therefore, Q^* will definitely be 1 because of the NAND gate's logic. Q^* was

fed back into α , and together with $\bar{S} = 1$, the output Q was 0 because of the NAND gate's logic.

$$\begin{aligned}\beta : Q^* &= 1 \\ \alpha : Q &= 0\end{aligned}$$

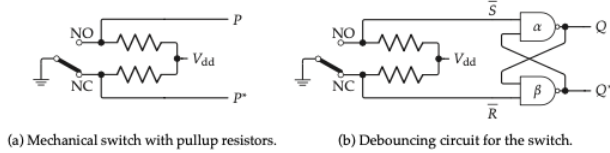


Figure 10: Two circuits for SPDT switch implemented in the lab. [Picone et al. \(2024\)](#).

3.4.3 Switch Pressed

When the switch was pressed, the circuit changed its state. The switch was now closed at NO and open at NC. This meant that V_{dd} flowed through the pull-down resistor to GND and therefore made $\bar{S} = 0$. Therefore, regardless of what the other input was for α , $Q = 1$, which in turn fed into β . At the same time, the switch was now closed at NC, so V_{dd} flowed through the pull-up resistor to $\bar{R} = 1$. Since $Q = 1$ and $\bar{R} = 1$, the output Q^* was 0 because of the NAND gate's logic.

$$\begin{aligned}\beta : Q^* &= 0 \\ \alpha : Q &= 1\end{aligned}$$

3.4.4 Switch Released

When the switch was released, the circuit returned to its initial state.

3.4.5 Debouncing vs Actual pressing of the switch

The NAND IC locked in the status of Q , which was also connected to DIO0 that was being monitored by the FPGA on the myRIO. The NAND flip-flop circuit ignored the noise as part of the lock-in mechanism. So even when the switch bounced, which affected Q in rapid succession, Q did not change. The NAND IC simply ignored the noise and held the last stable state.

The question then is, how does the NAND circuit distinguish between a legitimate switch press and a bounce? Could we not press the switch in rapid succession to simulate a bounce? For this, we look at what is defined by a legitimate switch press. The time interval for a legitimate switch press is typically longer than the duration of a bounce, allowing the NAND circuit to distinguish between the two. A human pressing the switch is 50 to 200 ms. As we saw from the ΔBX measurements in [Figure 8](#) and in [Figure 7](#), the bounces were measured in μs . The human press is many thousands of times longer than the bounce duration. So while it is theoretically possible to simulate a bounce via

rapid succession switch presses, we conclude that in reality, this is not physically possible, so there is no risk of encountering false negatives.

3.5 Problem L5.5

In your own words, explain how the main program thread configures the interrupt thread, how it communicates with the interrupt thread during execution, and how the interrupt thread functions.

We defined a global data structure (ThreadResource) to hold the necessary information for the interrupt, most importantly the IRQ number (irqNumber). We then declared an instance of ThreadResource in main() and gave the instance an arbitrary but unique irqNumber (in our case, irqNumber = 2). DIO0 was associated to the ThreadResource instance (irqThread0) and the irqNumber and was tasked to be triggered when a Falling Edge is detected on DIO0.

We then created a thread to monitor and catch irqNumber = 2, and when it catches this, it executes the corresponding interrupt service routine named DI_ISR. The code on what to do when the interrupt is received is defined in the DI_ISR function.

4 Author Contributions

There is only one author for this report and lab.

References

Rico A. R. Picone, Joseph L. Garbini, and Cameron N. Devine. *An introduction to real-time computing for mechanical engineers*. The MIT Press, 2024.